

CAPÍTULO 03 · GUÍA DEL ESTUDIANTE

Introducción real a Spring Boot y primer backend ejecutable

Guía de trabajo · Aplique el patrón a SU proyecto asignado

Programación Aplicada 2026-2 · Backend ParkFlow 360 · Java 21 + Spring Boot

Regla de transferencia: El profesor demostrará este capítulo con ParkFlow 360 y el par Cliente → Vehículo. Usted NO debe copiar esas entidades. Debe seguir el mismo proceso con el nombre de su propio sistema y con las dos entidades 1:N que seleccionó en los capítulos anteriores.

1. Qué logrará en este capítulo

- Crear el proyecto backend real de su sistema con Spring Boot, Maven y Java 21.
- Comprender qué es Spring Boot antes de utilizar JPA.
- Crear y ejecutar un endpoint REST.
- Comprender Bean, IoC e inyección de dependencias.
- Crear un endpoint demo relacionado con su entidad padre, todavía sin base de datos.
- Dejar el proyecto organizado para evolucionar luego a monolito hexagonal.

2. Antes de comenzar: complete esta ficha

Dato	Su respuesta
Nombre del sistema	_____
Nombre del backend	_____ -backend
Package base	com. _____
Entidad padre 1:N	_____
Entidad dependiente N	_____
Relación	1 _____ tiene muchos _____

Control de comprensión: Si todavía no puede explicar claramente cuál es su entidad padre y cuál la dependiente, no continúe. Revise su modelo físico y sus FK.

3. Crear el proyecto Spring Boot

En IntelliJ IDEA: File → New → Project → Spring Boot / Spring Initializr. Configure:

Programación Aplicada 2026-2 · Backend ParkFlow 360

Campo	Qué debe colocar
Name	<nombre-sistema>-backend
Language	Java
Type	Maven
Group	com.<organizacion-o-proyecto>
Artifact	<nombre-sistema>-backend
Package name	su package base
JDK / Java	21
Packaging	Jar

Seleccione solamente:

- Spring Web
- Validation
- Spring Boot DevTools (opcional)

Prohibido por ahora: No seleccione Spring Data JPA ni PostgreSQL Driver todavía. La base de datos ya existe, pero en este capítulo estamos aprendiendo Spring Boot y HTTP.

4. Verifique la estructura creada

```

<su-backend>/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/.../
│   │   │       └── SuAplicacionApplication.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
│       └── java/...

```

Debe poder explicar qué guarda cada carpeta antes de continuar.

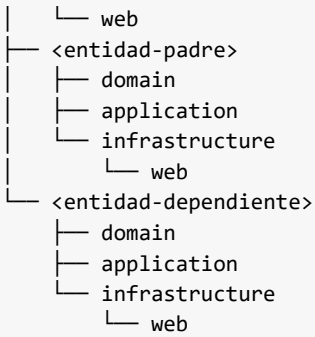
5. Cree packages compatibles con el futuro monolito hexagonal

Adapte los nombres <padre> y <dependiente> a sus dos entidades. Ejemplo del profesor: customer y vehicle.

```

com.<su-proyecto>
├── SuAplicacionApplication.java
├── shared
└── application

```



Convención del curso: No cree un package global llamado controller, otro service y otro repository para todo el sistema. El curso organizará el monolito por módulos de negocio.

6. Ejecute la aplicación sin escribir ningún endpoint

Ejecute la clase principal. En consola debe observar que la aplicación arranca y que el servidor web queda escuchando (normalmente en 8080).

```

@SpringBootApplication
public class SuAplicacionApplication {
    public static void main(String[] args) {
        SpringApplication.run(SuAplicacionApplication.class, args);
    }
}
  
```

No copie el nombre del profesor. Use el nombre generado para su sistema.

7. Construya su endpoint /api/health

Cree un controller en shared.web. Puede adaptar este patrón:

```

@RestController
@RequestMapping("/api/health")
public class HealthController {

    @GetMapping
    public Map<String, Object> health() {
        return Map.of(
            "status", "OK",
            "application", "NOMBRE-DE-SU-BACKEND",
            "timestamp", Instant.now().toString()
        );
    }
}
  
```

Pruebe:

```
GET http://localhost:8080/api/health
```

8. Cree su primer Bean de aplicación

Cree un servicio simple dentro de `shared.application`. No coloque lógica de persistencia.

```
@Service
public class ProjectInfoService {

    public String projectName() {
        return "NOMBRE DE SU PROYECTO";
    }

    public String backendStage() {
        return "SPRING_BOOT_BASE";
    }
}
```

Inyéctelo por constructor en `HealthController`. No utilice `new ProjectInfoService()` dentro del controller.

9. Cree un endpoint demo de SU entidad padre

El profesor usa `Cliente`. Usted debe usar su propia entidad padre. Cree primero un record de salida con 3 a 5 campos representativos.

```
public record EntidadPadreDemoResponse(
    Long id,
    String campo1,
    String campo2,
    String campo3
) {}
```

Luego un servicio temporal:

```
@Service
public class EntidadPadreDemoService {

    public EntidadPadreDemoResponse obtenerDemo() {
        return new EntidadPadreDemoResponse(
            1L,
            "valor-1",
            "valor-2",
            "valor-3"
        );
    }
}
```

Y un controller dentro del módulo de esa entidad:

```
@RestController
```

```

@RequestMapping("/api/<plural-entidad-padre>")
public class EntidadPadreDemoController {

    private final EntidadPadreDemoService service;

    public EntidadPadreDemoController(EntidadPadreDemoService service) {
        this.service = service;
    }

    @GetMapping("/demo")
    public EntidadPadreDemoResponse demo() {
        return service.obtenerDemo();
    }
}

```

Criterio de evaluación: Cambie nombres, campos, paquete y URL según su propio proyecto. Si entrega “ClienteDemoResponse” sin que su proyecto tenga clientes, evidencia que copió y no transfirió el patrón.

10. Debe poder explicar este flujo

```

Cliente HTTP
↓
GET /api/<entidad>/demo
↓
Spring MVC
↓
SuController
↓
SuDemoService (Bean)
↓
SuResponse record
↓
JSON HTTP 200

```

11. Conceptos que debe dominar

Concepto	Debe poder explicarlo con sus palabras
Spring Boot	_____
Bean	_____
IoC	_____
DI	_____
@RestController	_____
@Service	_____
Constructor injection	_____

@GetMapping

12. Pruebe desde IntelliJ HTTP Client

```
### Health
GET http://localhost:8080/api/health
Accept: application/json

### Demo entidad padre
GET http://localhost:8080/api/<plural-entidad-padre>/demo
Accept: application/json
```

13. Evidencia obligatoria

- Captura o demostración de la aplicación ejecutándose.
- Respuesta HTTP 200 de /api/health.
- Respuesta HTTP 200 del endpoint demo de su entidad padre.
- Estructura de packages visible en IntelliJ.
- Archivo requests.http con ambas pruebas.
- Commit Git del capítulo.

```
git add .
git commit -m "feat: bootstrap Spring Boot backend and first REST endpoints"
```

14. Errores que debe saber diagnosticar

Síntoma	Primera pregunta
La app no arranca	¿El JDK configurado es Java 21?
404	¿La URL y @RequestMapping coinciden?
Spring no inyecta un servicio	¿La clase tiene @Service y está bajo el package base?
Puerto 8080 ocupado	¿Hay otra app ejecutándose?
No compila un import	¿El package/clase está en la ubicación correcta?

15. Preguntas de defensa individual

1. Explique qué hace @SpringBootApplication.
2. Explique qué es un Bean usando una clase de su proyecto.
3. ¿Quién crea su DemoService?
4. ¿Por qué el controller recibe el servicio por constructor?
5. Describa el camino de su petición HTTP hasta obtener JSON.

6. ¿Por qué todavía no usamos PostgreSQL?
7. ¿Cuál es su entidad padre y cuál la dependiente?
8. ¿Cómo evolucionará este módulo hacia arquitectura hexagonal?

16. Definition of Done

- El backend real de su proyecto existe y compila.
- Usa Java 21 y Maven.
- El package base es coherente con el proyecto.
- /api/health funciona.
- El endpoint demo usa SU entidad padre.
- Existe al menos un Bean inyectado por constructor.
- No hay JPA ni conexión a PostgreSQL todavía.
- El trabajo está versionado en Git.
- Cada integrante puede defender el flujo sin leer código.